

MVP ENGENHARIA DE DADOS

AUTOR: LEANDRO JORDAO DE OLIVEIRA BARROS

DATA: 21/09/2026

MATRÍCULA: 4052025000893

DATASET: <https://portal.inmet.gov.br/dadoshistoricos> (2025-2024-2023)

“POSSO VIAJAR?”

Sumário

1 Introdução.....	3
2 Plataforma.....	3
3 Dados	4
4 Modelagem do Pipeline de Dados.....	5
4.1 Camada Bronze	5
4.2 Camada Prata	8
4.3 Camada Ouro	10
5 Modelagem Analítica: Esquema Estrela	12
6 Dicionário de Dados	15
7 Considerações Finais.....	18
8 Autoavaliação.....	22

1. Introdução – “A viagem...”

A escolha do período ideal para realizar uma viagem é uma decisão que envolve múltiplos fatores, entre os quais se destacam as condições climáticas, a temperatura média e a incidência de chuvas no destino escolhido. Essas variáveis exercem influência direta na experiência do viajante e, conseqüentemente, na satisfação com o serviço prestado por agentes de turismo.

Nesse contexto, torna-se relevante o desenvolvimento de ferramentas capazes de apoiar o processo de tomada de decisão, oferecendo informações precisas e de fácil acesso sobre o clima em diferentes localidades e períodos específicos. A disponibilização desses dados permite que agentes de viagens orientem seus clientes de forma mais assertiva, reduzindo incertezas e aumentando a qualidade das recomendações.

O presente trabalho propõe a criação de um **Mínimo Produto Viável (MVP)** voltado para agentes de viagens, com o objetivo de responder a três questões fundamentais:

- a) Há ocorrência de chuvas na cidade X durante o período da viagem?
- b) Qual é a temperatura média esperada para a cidade nesse intervalo?
- c) Entre duas cidades candidatas, qual delas apresenta melhores condições de acordo com os critérios estabelecidos pelo cliente?

Ao fornecer respostas claras e objetivas a essas perguntas, a ferramenta busca contribuir para a otimização do processo de planejamento de viagens, oferecendo suporte tecnológico que alia praticidade, confiabilidade e valor agregado ao serviço de consultoria turística.

2. Plataforma

Trabalho desenvolvido no “[Databricks Free Edition](#)”, com auxílio do Copilot para desenvolvimento dos códigos e das imagens dos dicionários de dados.

3. Dados

Os arquivos disponibilizados pelo Instituto Nacional de Meteorologia (INMET), página:<https://portal.inmet.gov.br/dadoshistoricos>, referentes aos anos de 2023, 2024 e 2025, contemplam dados históricos de todas as cidades do território nacional. Embora essa abrangência represente uma fonte valiosa de informações, o volume de dados torna o processo de análise e modelagem extremamente complexo e demandante em termos de tempo e recursos computacionais.

Considerando que o presente estudo tem como objetivo o desenvolvimento de um Mínimo Produto Viável (MVP), optou-se por restringir a aplicação inicial a um conjunto reduzido de cidades. Essa delimitação permite a construção de um protótipo funcional capaz de demonstrar a viabilidade da proposta, sem comprometer a qualidade metodológica. Além disso, a escolha por um escopo limitado possibilita a realização de testes mais ágeis e controlados, criando as bases para futuras expansões que poderão contemplar a totalidade dos dados disponíveis.

Dessa forma, o MVP será concebido como uma prova de conceito, validando a metodologia proposta e estabelecendo um caminho para o desenvolvimento de soluções mais abrangentes e escaláveis no campo da consultoria turística apoiada por dados meteorológicos.

4. Modelagem do Pipeline de Dados

O processo de modelagem do pipeline foi desenvolvido na plataforma Databricks, utilizando a arquitetura em camadas bronze, prata e ouro, prática consolidada em projetos de engenharia de dados por permitir organização, rastreabilidade e refinamento progressivo das informações.

4.1 Camada Bronze

Na primeira etapa, os arquivos históricos referentes aos anos de 2023, 2024 e 2025, disponibilizados pelo Instituto Nacional de Meteorologia (INMET), foram transferidos para a camada bronze, dentro do catálogo denominado mvpeneghariadedados. Nessa camada, foram armazenados os dados brutos das cidades selecionadas para o MVP, preservando sua integridade original e possibilitando uma leitura inicial para compreensão da estrutura e qualidade das informações.

Imagem 01 – Databricks

Criação Catalog mvpeneghariadedados e volumes bronze, prata e ouro

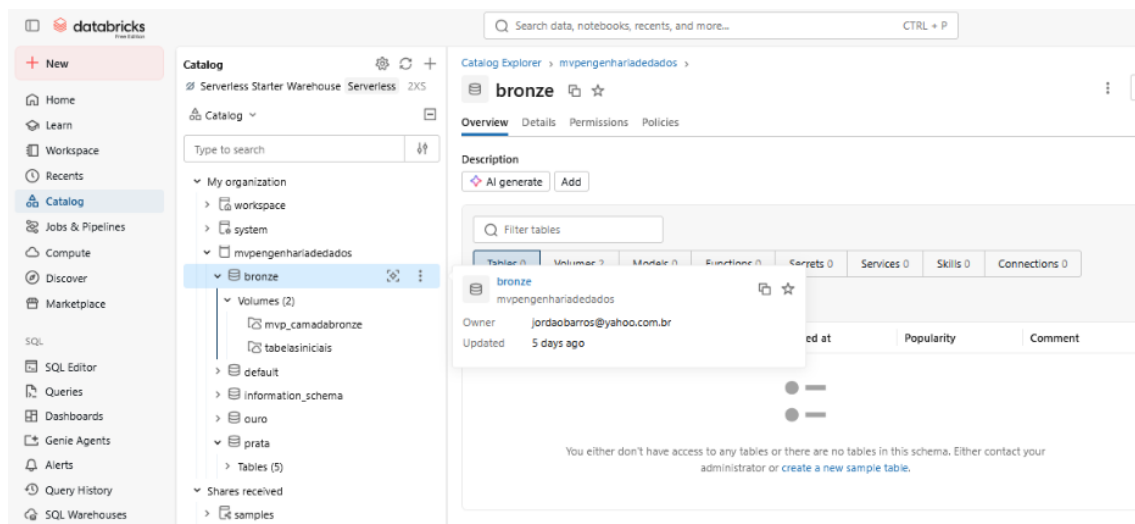
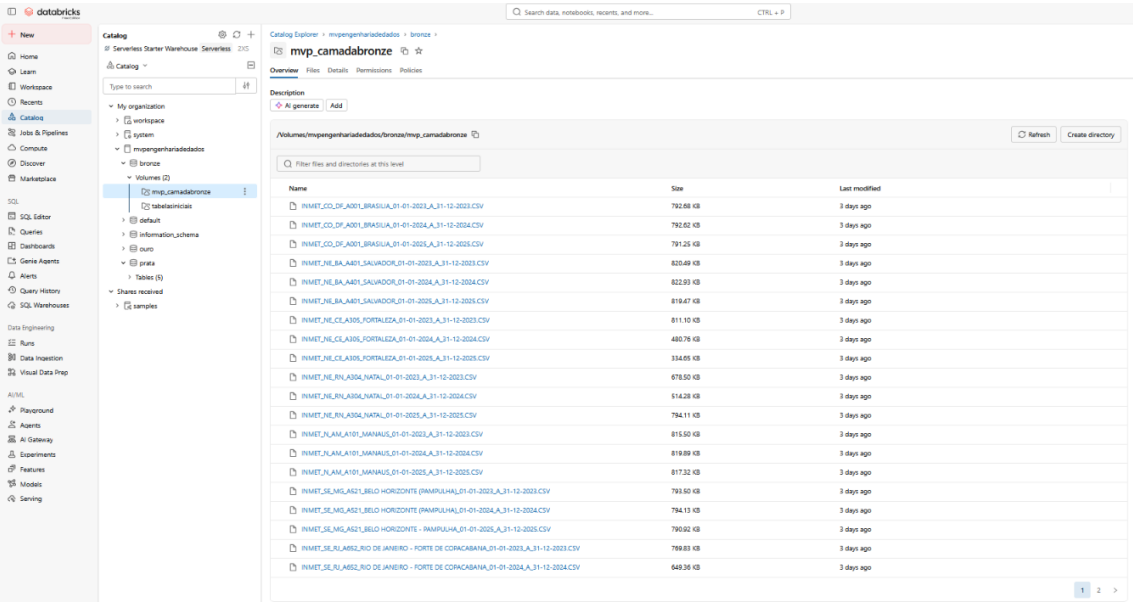


Imagem 02 – Arquivos em volume mvp_camadabronze



Notebook Bronze

Imagem 03: Tratamento Camada Bronze (TTO Bronze) parte 01

```
import codecs
import glob
from pyspark.sql.functions import col, when, to_date, year, month, dayofmonth, regexp_replace, lit

# Caminho da pasta com todos os CSVs
pasta = "/Volumes/mvpenhariadedados/bronze/mvp_camadabronze/*.CSV"

# Lista de arquivos
arquivos = glob.glob(pasta)

dfs = []

for arquivo in arquivos:
    # 1. Lê o cabeçalho manualmente para pegar a estação
    with codecs.open(arquivo, "r", "ISO-8859-1") as f:
        linhas = [next(f) for _ in range(8)]
        cidade = None
        for linha in linhas:
            if "ESTACAO" in linha.upper():
                cidade = linha.split(":")[1].strip()

    # 2. Lê os dados (pulando os metadados)
    df_bronze = (spark.read
                  .option("header", True)
                  .option("delimiter", ";")
                  .option("skipRows", 8)
                  .option("encoding", "ISO-8859-1")
                  .csv(arquivo))
```

Imagem 04: Tratamento Camada Bronze (TTO Bronze) parte 02

2026-09-18 16:41:24

Notebook TTO BRONZE × Notebook TTO PRATA Notebook OURO +

Help Python Full Last edit was 3 days ago

3 days ago (20s)1

```
# 3. Tratamento
df_silver = (df_bronze
    .withColumn("Data", to_date(col("Data"), "yyyy/MM/dd"))
    .withColumn("Temperatura", regexp_replace(col("TEMPERATURA DO AR - BULBO SECO, HORARIA (°C)"), ",", ".").cast("double"))
    .withColumn("Precipitacao", regexp_replace(col("PRECIPITAÇÃO TOTAL, HORÁRIO (mm)"), ",", ".").cast("double"))
    .withColumn("Choveu", when(col("Precipitacao") > 0, "sim").otherwise("nao"))
    .withColumn("Ano", year(col("Data")))
    .withColumn("Mes", month(col("Data")))
    .withColumn("Dia", dayofmonth(col("Data")))
    .withColumn("Cidade", lit(cidade)) # adiciona a estação como coluna fixa
    .select("Cidade", "Data", "Temperatura", "Precipitacao", "Choveu", "Ano", "Mes", "Dia")
)

dfs.append(df_silver)

# 4. Unifica todos os DataFrames em um só
df_final = dfs[0]
for d in dfs[1:]:
    df_final = df_final.unionByName(d)

# 5. Gravação na camada silver
df_final.write.mode("overwrite").saveAsTable("mvpengenhariadedados.prata.clima_cidadesMVP")
```

Imagem 05: Tratamento Camada Bronze (TTO Bronze) parte 03 – visualização “Tabela Bronze”

3 days ago (2s)2

```
# carregar a tabela salva
df_silver_loaded = spark.table("mvpengenhariadedados.prata.clima_cidadesmvp")

# mostrar os primeiros registros
display(df_silver_loaded.limit(20))
```

See performance (1)

df_silver_loaded: pyspark.sql.connect.dataframe.DataFrame = [Cidade: string, Data: date ... 6 more fields]

	Cidade	Data	1.2 Temperatura	1.2 Precipitacao	1.2 Choveu	1.2 Ano	1.2 Mes	1.2 Dia
1	:BRASILIA	2023-01-01	20.1	0	nao	2023	1	1
2	:BRASILIA	2023-01-01	19.2	0	nao	2023	1	1
3	:BRASILIA	2023-01-01	19.3	0	nao	2023	1	1
4	:BRASILIA	2023-01-01	19.3	0	nao	2023	1	1
5	:BRASILIA	2023-01-01	19.7	0	nao	2023	1	1
6	:BRASILIA	2023-01-01	19.1	0	nao	2023	1	1
7	:BRASILIA	2023-01-01	18.5	0	nao	2023	1	1
8	:BRASILIA	2023-01-01	18.5	0	nao	2023	1	1
9	:BRASILIA	2023-01-01	18.2	0	nao	2023	1	1
10	:BRASILIA	2023-01-01	18.1	0	nao	2023	1	1
11	:BRASILIA	2023-01-01	19	0	nao	2023	1	1
12	:BRASILIA	2023-01-01	19.4	0.2	sim	2023	1	1
13	:BRASILIA	2023-01-01	21	0	nao	2023	1	1
14	:BRASILIA	2023-01-01	22.5	0	nao	2023	1	1
15	:BRASILIA	2023-01-01	23.6	0	nao	2023	1	1

4.2 Camada Prata

Posteriormente, os dados foram tratados e refinados. A partir das “planilhas bronze”, realizou-se a seleção apenas das variáveis necessárias para atender aos objetivos do MVP. Foi criada uma tabela na camada prata, contendo os registros filtrados. Em seguida, procedeu-se à padronização dos nomes das cidades, de modo a evitar inconsistências e erros durante as consultas. Esse processo resultou em uma tabela consolidada, com nomenclatura uniforme e adequada para manipulação posterior.

Imagem 06: parte final Notebook Prata tratando os nomes das cidades



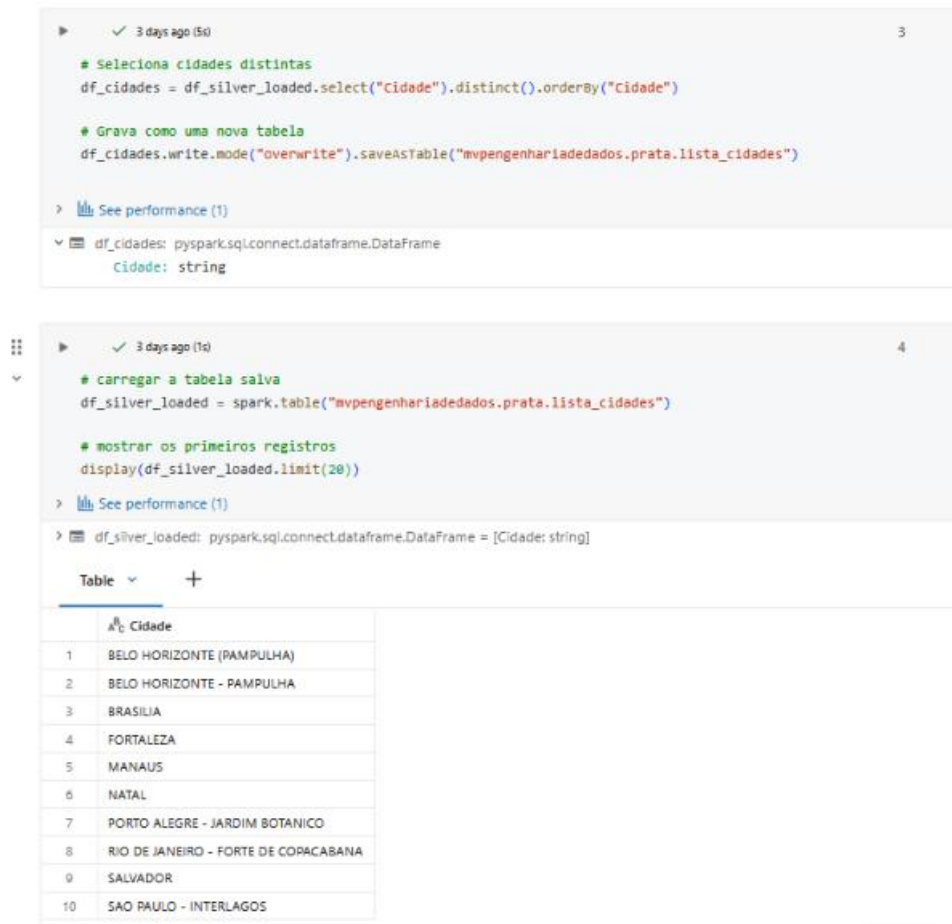
```
dfs.append(df_silver)

# 4. Unifica todos os DataFrames em um só
df_final = dfs[0]
for d in dfs[1:]:
    df_final = df_final.unionByName(d)
from pyspark.sql.functions import regexp_replace, trim, col

# 5. Limpeza dos nomes das cidades
df_final_limpo = (df_final
    .withColumn("Cidade", trim(regexp_replace(col("Cidade"), ";", "")))
)

# 6. Gravação na camada silver
df_final_limpo.write.mode("overwrite").saveAsTable("mvpengenhariadedados.prata.clima_cidadesMVP_limpa")
```

Imagem 07: verificando as Cidades



```
# Seleciona cidades distintas
df_cidades = df_silver_loaded.select("Cidade").distinct().orderBy("Cidade")

# Grava como uma nova tabela
df_cidades.write.mode("overwrite").saveAsTable("mvpengenhariadedados.prata.lista_cidades")

df_cidades: pyspark.sql.connect.dataframe.DataFrame
Cidade: string
```

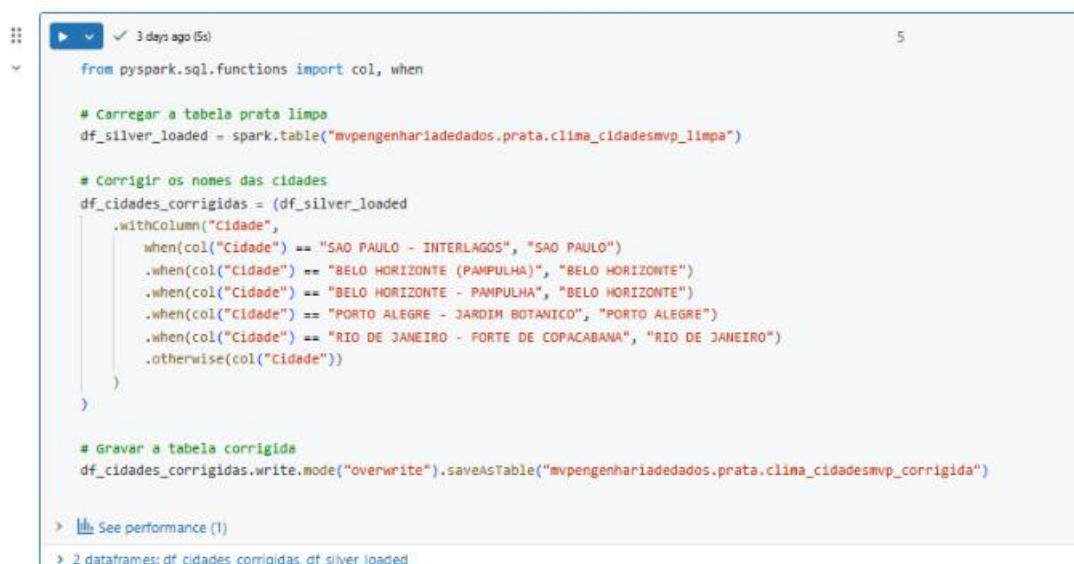
```
# carregar a tabela salva
df_silver_loaded = spark.table("mvpengenhariadedados.prata.lista_cidades")

# mostrar os primeiros registros
display(df_silver_loaded.limit(20))

df_silver_loaded: pyspark.sql.connect.dataframe.DataFrame = [Cidade: string]
```

	Cidade
1	BELO HORIZONTE (PAMPULHA)
2	BELO HORIZONTE - PAMPULHA
3	BRASILIA
4	FORTALEZA
5	MANAUS
6	NATAL
7	PORTO ALEGRE - JARDIM BOTANICO
8	RIO DE JANEIRO - FORTE DE COPACABANA
9	SALVADOR
10	SAO PAULO - INTERLAGOS

Imagem 08: Tratamento dos nomes das Cidades



```
from pyspark.sql.functions import col, when

# Carregar a tabela prata limpa
df_silver_loaded = spark.table("mvpengenhariadedados.prata.clima_cidadesmvp_limpa")

# Corrigir os nomes das cidades
df_cidades_corrigidas = (df_silver_loaded
    .withColumn("Cidade",
        when(col("Cidade") == "SAO PAULO - INTERLAGOS", "SAO PAULO")
        .when(col("Cidade") == "BELO HORIZONTE (PAMPULHA)", "BELO HORIZONTE")
        .when(col("Cidade") == "BELO HORIZONTE - PAMPULHA", "BELO HORIZONTE")
        .when(col("Cidade") == "PORTO ALEGRE - JARDIM BOTANICO", "PORTO ALEGRE")
        .when(col("Cidade") == "RIO DE JANEIRO - FORTE DE COPACABANA", "RIO DE JANEIRO")
        .otherwise(col("Cidade"))
    )
)

# Gravar a tabela corrigida
df_cidades_corrigidas.write.mode("overwrite").saveAsTable("mvpengenhariadedados.prata.clima_cidadesmvp_corrigida")

2 dataframes: df_cidades_corrigidas, df_silver_loaded
```

Imagem 09: Verificando o refinamento das cidades e criando a “tabela lista cidades final”

3 days ago (4d)7

```
# Seleciona cidades únicas da tabela corrigida
df_lista_cidades = df_corrigida.select("Cidade").distinct().orderBy("Cidade")

# Grava como nova tabela
df_lista_cidades.write.mode("overwrite").saveAsTable("mvpengenhariadedados.prata.lista_cidades_final")
```

See performance (1)

df_lista_cidades: pyspark.sql.connect.dataframe.DataFrame
Cidade: string

3 days ago (1d)8

```
# carregar a tabela salva
df_silver_loaded = spark.table("mvpengenhariadedados.prata.lista_cidades_final")

# mostrar os primeiros registros
display(df_silver_loaded.limit(20))
```

See performance (1)

df_silver_loaded: pyspark.sql.connect.dataframe.DataFrame = [Cidade: string]

Table +

	Cidade
1	BELO HORIZONTE
2	BRASILIA
3	FORTALEZA
4	MANAUS
5	NATAL
6	PORTO ALEGRE
7	RIO DE JANEIRO
8	SALVADOR
9	SAO PAULO

9 rows 1.422s runtime

4.3 Camada Ouro

Na camada ouro, foi desenvolvido um notebook denominado *Ouro*, responsável pela criação de widgets interativos. Esses widgets permitem ao usuário selecionar a cidade de interesse, bem como definir o período da viagem (dia inicial, dia final, mês inicial e mês final). A partir dessas entradas, o sistema retorna automaticamente um **dataframe** contendo três linhas, correspondentes aos anos de 2023, 2024 e 2025, com as seguintes informações:

- Cidade
- Ano
- Mês inicial e dia inicial
- Mês final e dia final
- Temperatura média do período

- Precipitação média do período
- Indicador binário de ocorrência de chuva

Imagem 10: Tela NOTEBOOK OURO

Camada Ouro – Consulta de Clima por Cidade

Este notebook faz parte da **camada ouro** do projeto. Ele funciona como uma ferramenta de pesquisa para agentes de viagens ou usuários que desejam consultar o histórico de clima por cidade e período.

Como usar:

1. Selecione a cidade no **dropdown**.
2. Informe datas inicial e final nos **widgets**.
3. Execute a célula de consulta para ver médias de temperatura, precipitação e chuva nos últimos anos.

3 days ago (1)

2

Widgets em Python dbutils.widgets.dropdown("Cidade", "BRASILIA", [row["Cidade" ...

> See performance (1)

09:38 AM (3)

3

cidade = dbutils.widgets.get("Cidade") mes_inicial = dbutils.widgets.get("Mes_In ...

> See performance (1)

Cidade	Ano	Mes_Inicial	Dia_Inicial	Mes_Final	Dia_Final	Temp_Media_Periodo	Precipitacao_Media_Periodo	Choveu_Periodo
SALVADOR	2023	7	3	7	15	24.33	0.13	sim
SALVADOR	2024	7	3	7	15	24.77	0.12	sim
SALVADOR	2025	7	3	7	15	23.61	0.26	sim

Imagem 11: Criação dos Widgets

```

# Widgets em Python
dbutils.widgets.dropdown("Cidade", "BRASILIA", [row["Cidade"] for row in spark.table("mpengenhariadados.prata.lista_cidades_final").select("Cidade").distinct().collect()])
dbutils.widgets.text("Mes_Inicial", "4")
dbutils.widgets.text("Dia_Inicial", "31")
dbutils.widgets.text("Mes_Final", "5")
dbutils.widgets.text("Dia_Final", "4")

```

Imagem 12: Consulta dos dados lançados nos widgets

FileEditViewRunHelpPythonFullLast edit was 3 days ago

Cidade

Dia_Final

Dia_Inicial

Mes_Final

Mes_Inicial

SALVADOR

12

23

2

1

09:38 AM (39s)

3

```
cidade = dbutils.widgets.get("Cidade")
mes_inicial = dbutils.widgets.get("Mes_Inicial")
dia_inicial = dbutils.widgets.get("Dia_Inicial")
mes_final = dbutils.widgets.get("Mes_Final")
dia_final = dbutils.widgets.get("Dia_Final")

query = f"""
SELECT
    Cidade,
    Ano,
    {mes_inicial} AS Mes_Inicial,
    {dia_inicial} AS Dia_Inicial,
    {mes_final} AS Mes_Final,
    {dia_final} AS Dia_Final,
    ROUND(AVG(Temperatura), 2) AS Temp_Media_Periodo,
    ROUND(AVG(Precipitacao), 2) AS Precipitacao_Media_Periodo,
    CASE WHEN SUM(CASE WHEN Choveu = 'sim' THEN 1 ELSE 0 END) > 0 THEN 'sim' ELSE 'nao' END AS Choveu_Periodo
FROM mvpengharladedados.prata.clima_cidadesmvp_corrigida
WHERE Cidade = '{cidade}'
    AND (
        (Mes = {mes_inicial} AND Dia >= {dia_inicial})
        OR (Mes = {mes_final} AND Dia <= {dia_final})
    )
GROUP BY Cidade, Ano
ORDER BY Ano
"""

df_resultado = spark.sql(query)
df_resultado.show()
```

Imagem 13: Dataframe final

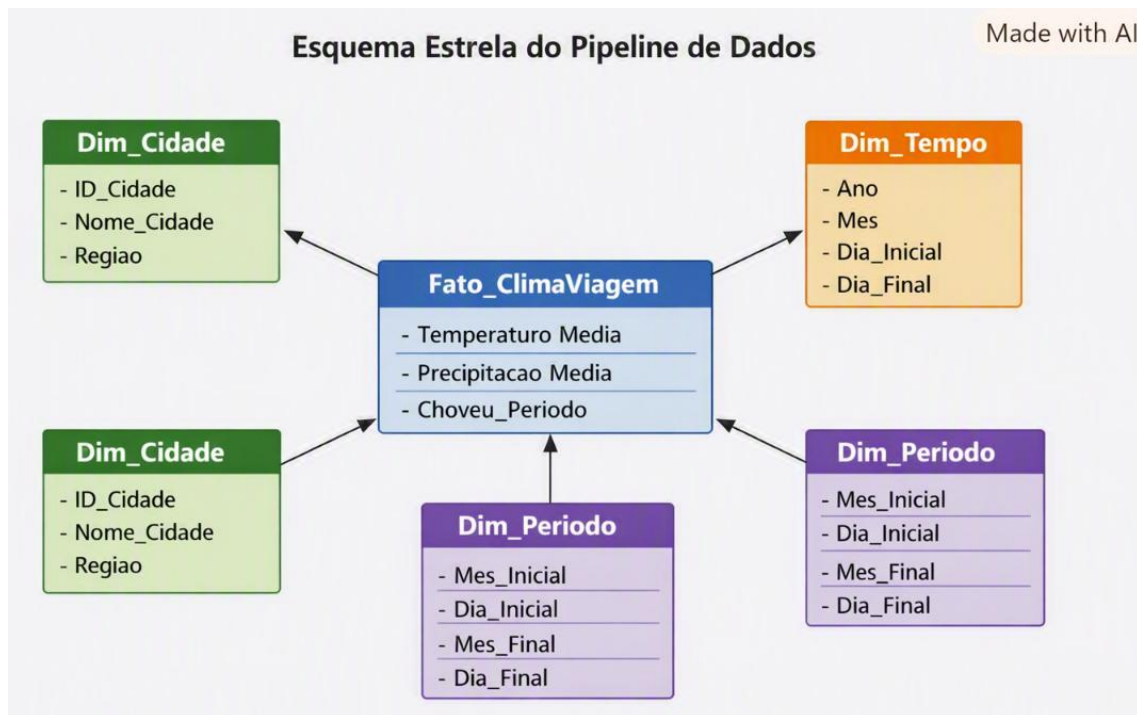
> df_resultado: pyspark.sql.connect.dataframe.DataFrame = [Cidade: string, Ano: integer ... 7 more fields]

Cidade	Ano	Mes_Inicial	Dia_Inicial	Mes_Final	Dia_Final	Temp_Media_Periodo	Precipitacao_Media_Periodo	Choveu_Periodo
SALVADOR	2023	7	3	7	15	24.33	0.13	sim
SALVADOR	2024	7	3	7	15	24.77	0.12	sim
SALVADOR	2025	7	3	7	15	23.61	0.25	sim

5. Modelagem Analítica: Esquema Estrela

Para complementar o pipeline desenvolvido no Databricks, foi adotada a modelagem analítica em **esquema estrela**, amplamente utilizada em projetos de Business Intelligence e Data Warehousing. Essa abordagem permite organizar os dados de forma a otimizar consultas, facilitar a interpretação das informações e garantir maior desempenho em análises exploratórias.

Imagem 14: Diagrama Estrela



No contexto do presente MVP, o esquema estrela foi estruturado da seguinte forma:

- **Tabela Fato (Fato_ClimaViagem)** Contém os indicadores numéricos e variáveis de interesse para análise, derivados da camada ouro. Entre as principais métricas estão:
 - Temperatura média do período
 - Precipitação média do período
 - Indicador binário de ocorrência de chuva
- **Tabelas Dimensão**
 - **Dim_Cidade:** informações sobre as cidades selecionadas (nome padronizado, código identificador, região).
 - **Dim_Tempo:** estrutura temporal detalhada (ano, mês, dia inicial, dia final), permitindo análises por diferentes granularidades.
 - **Dim_Período:** intervalo definido pelo usuário para a viagem, possibilitando consultas comparativas entre períodos distintos.

Essa modelagem possibilita que o agente de viagens, ou qualquer usuário da ferramenta, realize consultas de forma intuitiva, como por exemplo:

- Comparar a temperatura média entre diferentes cidades em um mesmo período.

- Verificar a incidência de chuvas em anos distintos para uma mesma localidade.
- Avaliar a evolução climática de uma cidade ao longo dos anos disponíveis (2023, 2024 e 2025).

O esquema estrela, aliado ao pipeline em camadas bronze, prata e ouro, garante não apenas a integridade e qualidade dos dados, mas também a eficiência analítica necessária para que o MVP seja escalável e possa futuramente incorporar a totalidade dos dados disponibilizados pelo INMET.

6. Dicionário de Dados

a. Tabela Bronze “INMET”

6.1.1 Tabela Bronze “INMET” – parte 01

Nome do Campo	Tipo de Dado	Descrição
Data	string	Data da coleta da observação meteorológica.
Hora UTC	string	Horário da medição em tempo universal coordenado.
PRECIPITACAO TOTAL, HORARIO (mm)	string	Quantidade total de precipitação registrada no período horário.
PRESSAO ATMOSFERICA AO NIVEL DA ESTACAO, HORARIA (mB)	string	Pressão atmosférica medida ao nível da estação meteorológica.
PRESSAO ATMOSFERICA MAX.NA HORA ANT. (AUT) (mB)	string	Valor máximo da pressão atmosférica na hora anterior.
PRESSAO ATMOSFERICA MIN. NA HORA ANT. (AUT) (mB)	string	Valor mínimo da pressão atmosférica na hora anterior.
RADIACAO GLOBAL (Kj/m²)	string	Quantidade de energia solar incidente por metro quadrado.

6.1.2 Tabela Bronze “INMET” – parte 02

Nome do Campo	Tipo de Dado	Descrição
TEMPERATURA DO AR - BULBO SECO, HORARIA (°C)	string	Temperatura do ar medida pelo bulbo seco, em graus Celsius.
TEMPERATURA DO PONTO DE ORVALHO (°C)	string	Temperatura na qual o vapor d'água se condensa, em graus Celsius.
TEMPERATURA MAXIMA NA HORA ANT. (AUT) (°C)	string	Temperatura máxima registrada na hora anterior, em graus Celsius.
TEMPERATURA MINIMA NA HORA ANT. (AUT) (°C)	string	Temperatura mínima registrada na hora anterior, em graus Celsius.
TEMPERATURA ORVALHO MAX. NA HORA ANT. (AUT) (°C)	string	Temperatura máxima do ponto de orvalho registrada na hora anterior, em graus Celsius.
TEMPERATURA ORVALHO MIN. NA HORA ANT. (AUT) (°C)	string	Temperatura mínima do ponto de orvalho registrada na hora anterior, em graus Celsius.

6.1.3 Tabela Bronze “INMET” – parte 03

Nome do Campo	Tipo de Dado	Descrição
UMIDADE REL. MAX. NA HORA ANT. (AUT) (%)	string	Umidade relativa máxima registrada na hora anterior, em porcentagem.
UMIDADE REL. MIN. NA HORA ANT. (AUT) (%)	string	Umidade relativa mínima registrada na hora anterior, em porcentagem.
UMIDADE RELATIVA DO AR, HORARIA (%)	string	Umidade relativa do ar medida no período horário, em porcentagem.
VENTO, DIRECAO HORARIA (gr) (° (gr))	string	Direção do vento em graus, medida no período horário.
VENTO, RAJADA MAXIMA (m/s)	string	Velocidade máxima das rajadas de vento registradas, em metros por segundo.
VENTO, VELOCIDADE HORARIA (m/s)	string	Velocidade média do vento no período horário, em metros por segundo.

b. Tabela “clima_cidadesmvp_corrigida”

Nome do Campo	Tipo de Dado	Descrição
Cidade	string	Nome da cidade padronizado após o tratamento dos dados na camada bronze.
Data	date	Data correspondente à observação meteorológica, convertida para formato de data.
Temperatura	double	Valor numérico da temperatura média registrada no período.
Precipitacao	double	Valor numérico da precipitação total registrada no período.
Choveu	string	Indicador textual que informa se houve ocorrência de chuva (“sim” ou “nao”).
Ano	integer	Ano extraído da variável Data, utilizado para agregações e comparações temporais.
Mes	integer	Mês extraído da variável Data, utilizado para análises sazonais.
Dia	integer	Dia extraído da variável Data, permitindo granularidade diária nas consultas.

c. Tabela “lista_cidades_final”

Nome do Campo	Tipo de Dado	Descrição
Cidade	string	Nome padronizado da cidade, utilizado como chave de referência para as consultas e agregações nas demais tabelas do pipeline.

d. Dataframe Final

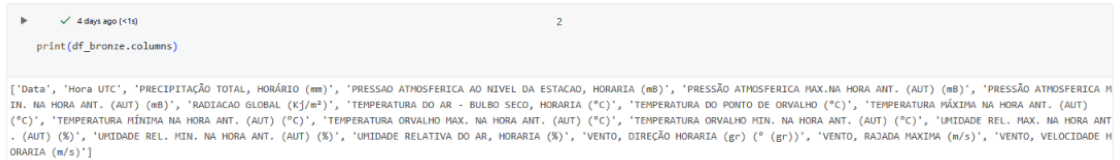
Nome do Campo	Tipo de Dado	Descrição
Cidade	string	Nome da cidade selecionada pelo usuário.
Ano	integer	Ano da observação (2023, 2024 ou 2025).
Mes_Inicial	integer	Mês inicial do período definido pelo usuário.
Dia_Inicial	integer	Dia inicial do período definido pelo usuário.
Mes_Final	integer	Mês final do período definido pelo usuário.
Dia_Final	integer	Dia final do período definido pelo usuário.
Temp_Media_Periodo	double	Temperatura média calculada para o período selecionado.
Precipitacao_Media_Periodo	double	Precipitação média calculada para o período selecionado.
Choveu_Periodo	string	Indicador textual que informa se houve ocorrência de chuva no período (“sim” ou “nao”).

7. Considerações Finais

a. Dados:

Os datasets do INMET são bem extensos, pois mensuram dados nas estações durante todos os dias do ano, durante o dia todo em horários específicos.

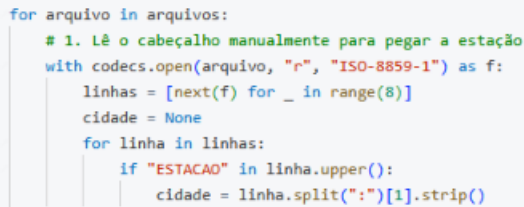
Imagem 15 – Colunas dos datasets INMET



No entanto, apesar de pesados estão bem completos e o tratamento dos dados geraram duas questões:

1. Os metadados dos datasets estão nas primeiras linhas do arquivo e precisamos buscar as “Estações” que de fato representam as cidades que definimos em nossas consultas.

Imagem 16 – Buscando as Estações nos metadados do .csv

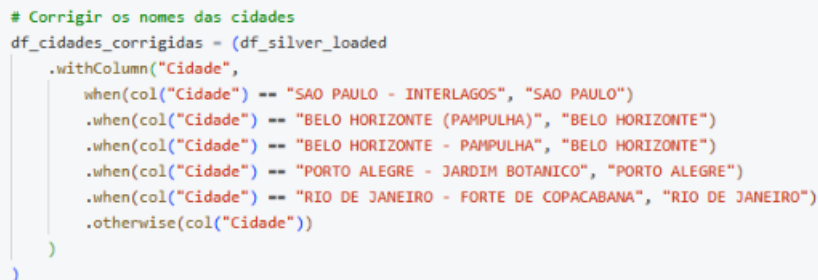


2. Ajuste dos nomes das CIDADES para que não houvesse erro na seleção ou input de dados nos widgets do notebook ou no.

Imagem 17 – limpando “;”

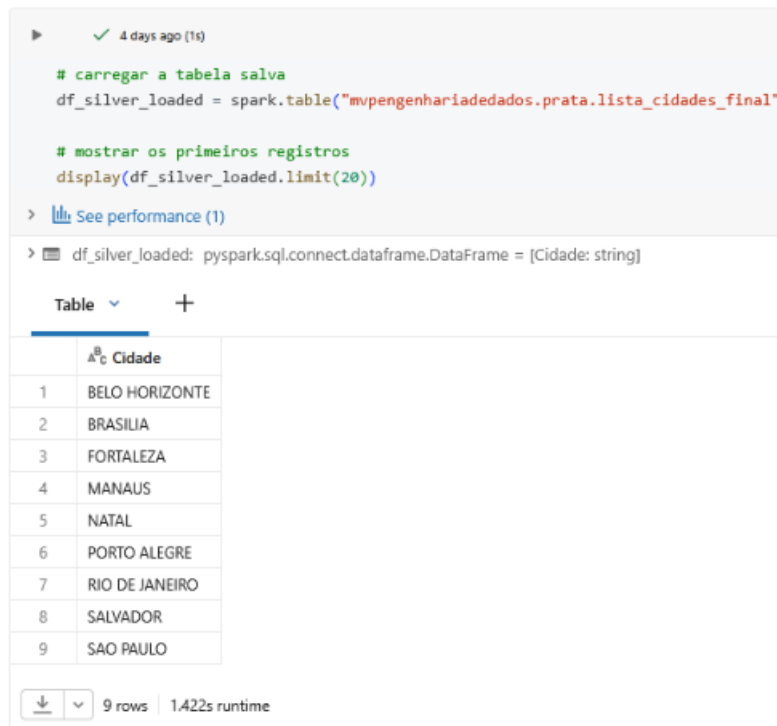


Imagem 18 – corrigindo os nomes das cidades



Desta forma, podemos citar estes como os nossos principais ruídos no tratamento dos datasets para ajuste das cargas e criação da tabela final para as consultas com os parâmetros que precisávamos:

Imagem 19 – Lista das cidades



```

# carregar a tabela salva
df_silver_loaded = spark.table("mvpengenhariadedados.prata.lista_cidades_final")

# mostrar os primeiros registros
display(df_silver_loaded.limit(20))

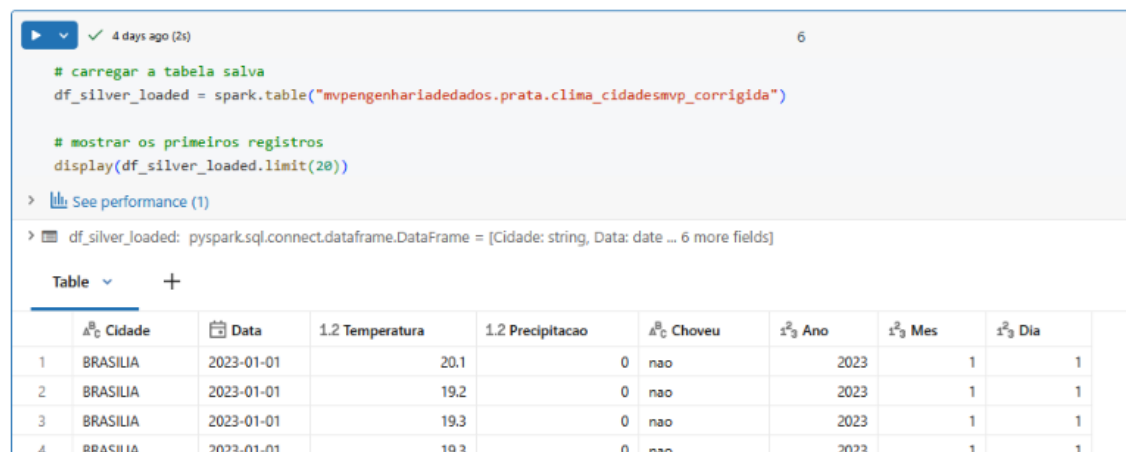
```

df_silver_loaded: pyspark.sql.connect.dataframe.DataFrame = [Cidade: string]

	Cidade
1	BELO HORIZONTE
2	BRASILIA
3	FORTALEZA
4	MANAUS
5	NATAL
6	PORTO ALEGRE
7	RIO DE JANEIRO
8	SALVADOR
9	SAO PAULO

9 rows | 1.422s runtime

Imagem 20 – tabela final para consulta “clima_cidadesmvp_corrigida”



```

# carregar a tabela salva
df_silver_loaded = spark.table("mvpengenhariadedados.prata.clima_cidadesmvp_corrigida")

# mostrar os primeiros registros
display(df_silver_loaded.limit(20))

```

df_silver_loaded: pyspark.sql.connect.dataframe.DataFrame = [Cidade: string, Data: date ... 6 more fields]

	Cidade	Data	1.2 Temperatura	1.2 Precipitacao	1.2 Choveu	1.2 Ano	1.2 Mes	1.2 Dia
1	BRASILIA	2023-01-01	20.1	0	nao	2023	1	1
2	BRASILIA	2023-01-01	19.2	0	nao	2023	1	1
3	BRASILIA	2023-01-01	19.3	0	nao	2023	1	1
4	BRASILIA	2023-01-01	19.3	0	nao	2023	1	1

b. Consulta (Ferramenta) Notebook Ouro

Colocamos uma célula markdown/texto para que qualquer usuário conseguisse fazer a consulta apenas digitando as informações solicitados pelos widgets:

Imagem 21 – célula texto(markdown)

Camada Ouro – Consulta de Clima por Cidade

Este notebook faz parte da **camada ouro** do projeto.

Ele funciona como uma ferramenta de pesquisa para agentes de viagens ou usuários que desejam consultar o histórico de clima por cidade e período.

Como usar:

1. Selecione a cidade no **dropdown**.
2. Informe datas inicial e final nos **widgets**.
3. Execute a célula de consulta para ver médias de temperatura, precipitação e chuva nos últimos anos.

Imagem 22 – widgets

HomeNew Notebook 2026-09-18 16:41:24Notebook TTO BRONZENotebook TTO PRATANotebook OURO

FileEditViewRunHelpPythonFullLast edit was 3 minutes ago

Cidade

Dia_Final

Dia_Inicial

Mes_Final

Mes_Inicial

SALVADOR

12

23

2

1

c. Respondendo as perguntas do MVP:

a) Há ocorrência de chuvas na cidade X durante o período da viagem?

Imagem 23 – consulta cidade de Salvador

Cidade	Ano	Mes_Inicial	Dia_Inicial	Mes_Final	Dia_Final	Temp_Media_Periodo	Precipitacao_Media_Periodo	Choveu_Periodo
SALVADOR	2023	7	3	7	15	24.33	0.13	sim
SALVADOR	2024	7	3	7	15	24.77	0.12	sim
SALVADOR	2025	7	3	7	15	23.61	0.25	sim

b) Qual é a temperatura média esperada para a cidade nesse intervalo?

c) Entre duas cidades candidatas, qual delas apresenta melhores condições de acordo com os critérios estabelecidos pelo cliente?

Imagem 24 – consulta cidade de Porto Alegre

Cidade	Ano	Mes_Inicial	Dia_Inicial	Mes_Final	Dia_Final	Temp_Media_Periodo	Precipitacao_Media_Periodo	Choveu_Periodo
PORTO ALEGRE	2023	7	3	7	15	15.46	0.28	sim
PORTO ALEGRE	2024	7	3	7	15	13.74	0.1	sim
PORTO ALEGRE	2025	7	3	7	15	13.08	0.12	sim

Imagem 25 – consulta cidade de São Paulo

Cidade	Ano	Mes_Inicial	Dia_Inicial	Mes_Final	Dia_Final	Temp_Media_Periodo	Precipitacao_Media_Periodo	Choveu_Periodo
SAO PAULO	2023	7	3	7	15	17.06	0.02	sim
SAO PAULO	2024	7	3	7	15	16.44	0.1	sim
SAO PAULO	2025	7	3	7	15	14.7	0.01	sim

Imagem 26 – consulta cidade de Fortaleza

Cidade	Ano	Mes_Inicial	Dia_Inicial	Mes_Final	Dia_Final	Temp_Media_Periodo	Precipitacao_Media_Periodo	Choveu_Periodo
FORTALEZA	2023	7	3	7	15	27.09	0.03	sim
FORTALEZA	2024	7	3	7	15	28.88	0.0	nao
FORTALEZA	2025	7	3	7	15	NULL	NULL	nao

A partir da tabela/dataframe de retorno da nossa consulta conseguimos sugerir uma previsão de como será o período da viagem solicitado pelo cliente.

Imagem 27 - consulta

```
query = f"""
SELECT
    Cidade,
    Ano,
    {mes_inicial} AS Mes_Inicial,
    {dia_inicial} AS Dia_Inicial,
    {mes_final} AS Mes_Final,
    {dia_final} AS Dia_Final,
    ROUND(AVG(Temperatura), 2) AS Temp_Media_Periodo,
    ROUND(AVG(Precipitacao), 2) AS Precipitacao_Media_Periodo,
    CASE WHEN SUM(CASE WHEN Choveu = 'sim' THEN 1 ELSE 0 END) > 0 THEN 'sim' ELSE 'nao' END AS Choveu_Periodo
FROM mvpengenhariadados.prata.clima_cidadesmvp_corrigida
WHERE Cidade = '{cidade}'
    AND (
        (Mes = {mes_inicial} AND Dia >= {dia_inicial})
        OR (Mes = {mes_final} AND Dia <= {dia_final})
    )
GROUP BY Cidade, Ano
ORDER BY Ano
"""

df_resultado = spark.sql(query)
df_resultado.show()
```

Desta forma, todas as perguntas do MVP foram respondidas com base no histórico dos últimos três anos. A primeira pergunta é respondida com as colunas “Choveu_Periodo” e “Precipitação_Media_Período”. A segunda pergunta está na coluna “Temp_Media_Periodo”. A terceira que solicita uma comparação entre duas ou mais cidades, a fim de atender a demanda do cliente, pode ser feita consultando várias cidades para o mesmo período, como nos exemplos demonstrados nas imagens 24, 25 e 26, onde numa possível viagem buscando não encontrar a chuva e ter temperaturas mais altas no mês de julho, a cidade de Fortaleza seria a mais indicada quando comparada a São Paulo e Porto Alegre.

Estes dados poderiam ser ainda mais precisos, aumentando o histórico de dados e/ou expandindo o número de cidades, pois a base de dados possui dados de todas as cidades no Brasil com estações do Instituto Nacional de Meteorologia -INMET.

8. Autoavaliação

O curso apresentou uma grande quantidade de conteúdo em um tempo relativamente curto, o que tornou desafiador absorver todo o conhecimento em profundidade e dominar plenamente as ferramentas voltadas para engenharia de dados. Ainda assim, foi possível compreender a direção a seguir e identificar onde e como buscar conhecimento adicional para aprimorar meu desenvolvimento.

Como estou em processo de migração de área e iniciando do zero na área de tecnologia da informação, seria inviável finalizar o curso já dominando os códigos em detalhe enquanto concilio trabalho e estudos. No entanto, consegui construir uma base sólida para continuar evoluindo.

A utilização da inteligência artificial, especialmente o Copilot, foi fundamental para a conclusão do trabalho, servindo como apoio na compreensão de conceitos e na execução prática das atividades. Essa experiência reforçou a importância de buscar constantemente novas fontes de aprendizado e de utilizar ferramentas que potencializem o desenvolvimento profissional.